

Meeting Assistant

Roadmap & Future Work

Document 3 of 3 · Product roadmap, advanced capabilities, and research directions · [Project overview](#) · [Implementation](#) · [CHANGELOG](#)

This document gathers everything that comes *after* the MVP: the product roadmap by sprint, three advanced differentiating capabilities (facilitator guide, document alignment, sentiment analysis), the operational quality cycle (MLOps), and a set of longer-horizon research ideas. It complements the "future ideas" in [Document 1](#) (vision) and [Document 2](#) (current system).

Status guide. Sprint 1 (the MVP) is complete. Among advanced capabilities, **facilitator guide** is already implemented (`GET /api/meetings/{id}/facilitation`); document alignment and sentiment analysis are designed but not yet built.

Permanently out of scope: Live streaming transcription during meetings and professional speaker diarization. Single-pass Gemini transcription from uploaded/recorded audio covers product needs without a separate STT pipeline.

Contents

1. [Roadmap at a Glance](#)
2. [Sprint 2 — Archive & Organizational Search](#)
3. [Sprint 3 — Jira & Operational Workflow](#)
4. [Sprint 4 — Users, Workspaces & Calendar](#)
5. [Sprint 5 — Scale, Quality & Deployment](#)
6. [Sprint 6+ — Organizational Intelligence Product](#)
7. [Advanced Capabilities \(Differentiators\)](#)
 1. [Facilitator Guide](#)

2. SOW / Contract Alignment
3. Sentiment Analysis
8. 8. MLOps & Quality Cycle
9. 9. Prioritization
10. 10. Technical Dependencies
11. 11. Longer-Horizon Research Ideas

1. Roadmap at a Glance

Sprint 1 

MVP دموی



Sprint 2

چند جلسه RAG آرشیو و

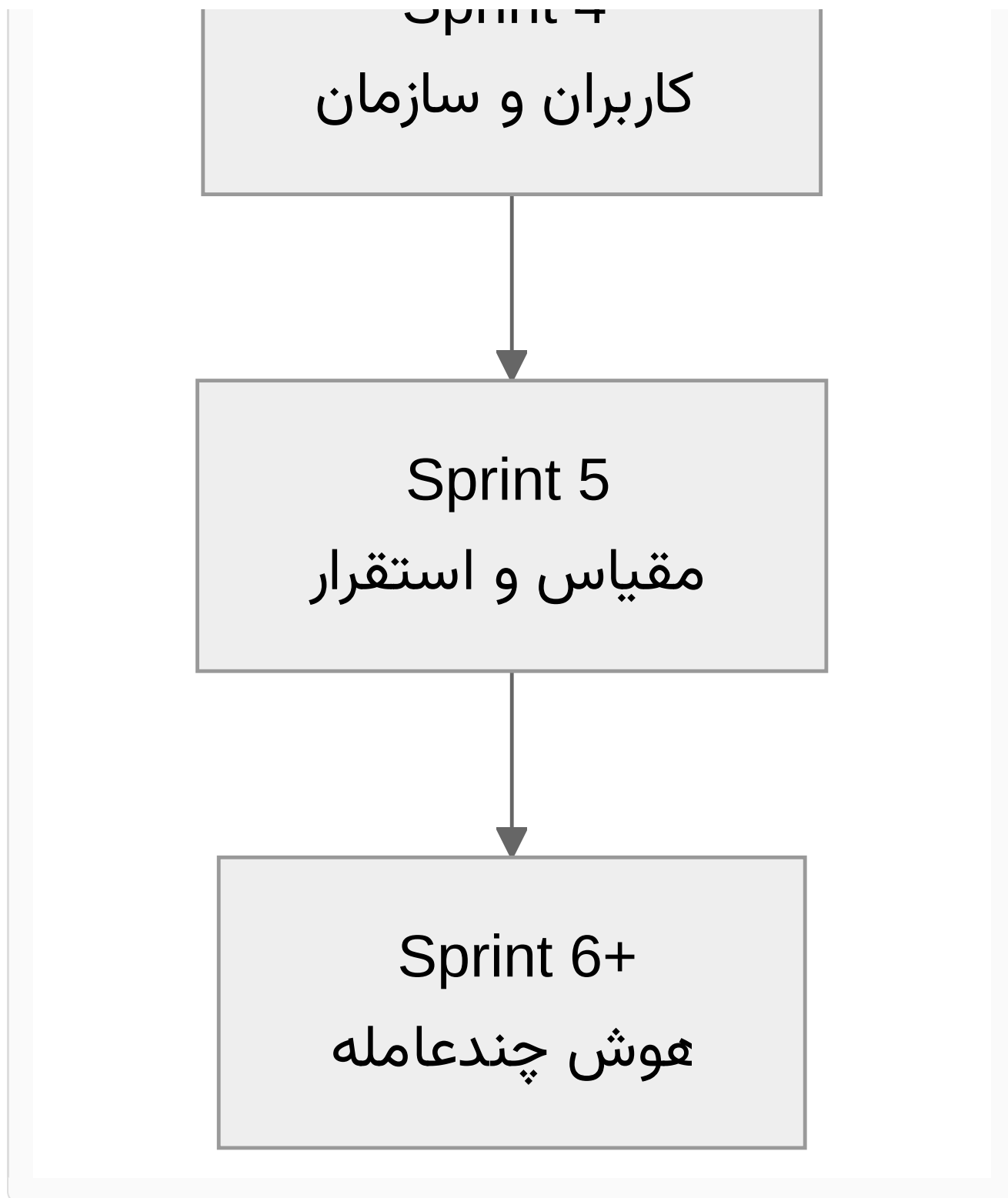


Sprint 3

هوشمند و یکپارچه سازی Jira



Sprint 4



The roadmap arc is deliberate: from one meeting (MVP) to organizational meeting memory (Sprint 2), to real operational workflow (Sprint 3), to a truly multi-tenant product (Sprints 4 and 5), and finally to differentiated multi-agent intelligence (Sprint 6+).

2. Sprint 2 — Archive & Organizational Search

Goal: Move from "one meeting" to "organizational meeting memory."

Capability	Description	Proposed implementation
Multi-meeting RAG	Query across meetings with date, project, participant filters.	Chroma metadata: <code>org_id</code> , <code>project</code> , <code>date</code> ; filter in query.
Archive page	Meeting list with full-text search on title/summary.	SQLite FTS5 on <code>title</code> , <code>summary</code> .
Tags & project	Categorize meetings (standup, planning, review).	<code>tags</code> , <code>project_key</code> columns in SQLite.
Summary export	Export to Markdown / PDF for sharing with managers.	Astro template or WeasyPrint from same <code>MeetingAnalysis</code> .
Meeting comparison	"Last sprint's decisions vs now" — agent over multiple <code>meeting_ids</code> .	<code>compare_agent</code> + top-k from each meeting.

Acceptance criteria

- Three meetings ingested; one cross-meeting question with citations from at least two meetings.
- Archive search under 500ms (up to ~100 meetings).
- Export one meeting summary without RTL errors.

3. Sprint 3 — Jira & Operational Workflow

Goal: Tasks actually reach the right people in Jira, not anonymous issues.

Capability	Description	Technical
Assignee mapping	"Sara" in transcript ← one <code>accountId</code> in Jira.	<code>speaker_jira_map</code> table; settings UI (ready).
<code>jira_agent</code>	Preview, deduplication, epic/link suggestions.	Pydantic AI tools: <code>search_existing_issues</code> ,

		<code>preview_create</code> .
Jira OAuth	Replace personal token — team-friendly.	Atlassian OAuth 2.0; encrypted refresh token storage.
Edit before submit	User edits tasks in UI, then bulk create.	Temporary state; POST only after confirmation.
Sync status	Show issue key and Jira link on meeting page.	<code>jira_keys_json</code> per task.
Issue template	Custom project fields (story points, component).	Map from <code>JIRA_PROJECT_KEY</code> + YAML config.

4. Sprint 4 — Users, Workspaces & Calendar

Goal: Real multi-user product and connection to everyday tools.

Capability	Description	Technical
Authentication	Email/password or org SSO login.	JWT session; or OIDC (Keycloak / Google Workspace).
Workspace	Each org's data isolated — meetings, speaker map, Jira.	<code>org_id</code> on all tables; tenancy middleware.
Roles (RBAC)	admin · editor · viewer — who can Jira create.	<code>memberships</code> table.
Calendar (metadata)	Import title/time/attendees from Google Calendar — transcript still paste/upload.	Calendar API read-only; pre-fill meeting form.
Summary email	Auto-send summary + meeting link to participants.	SMTP or Resend/Cloudflare Email; RTL HTML template.

Slack/Teams alerts	Short message: N new tasks + link.	Incoming webhook.
--------------------	------------------------------------	-------------------

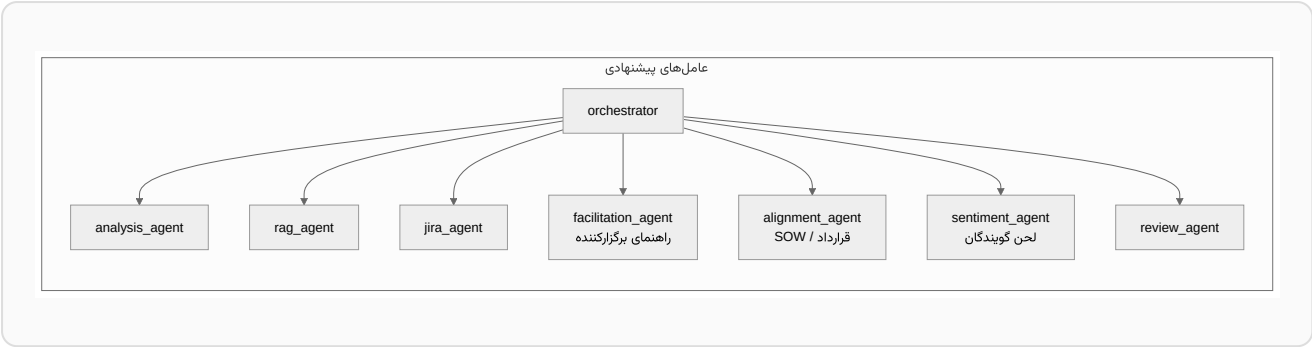
5. Sprint 5 — Scale, Quality & Deployment

Goal: Ready for organizational demo and production monitoring.

Capability	Description	Technical
PostgreSQL	Replace SQLite for multi-worker and backup.	SQLAlchemy + Alembic migration from current schema.
Vector at scale	Thousands of chunks without slowdown.	pgvector or Chroma Cloud; retention policy.
Ingest queue	Long meeting analysis in background.	Redis + worker (Celery / ARQ); status polling in UI.
Observability	Track Gemini latency and Jira errors.	OpenTelemetry + structured logs; one dashboard.
Deployment	Docker Compose; optional Cloud Run / VPS.	frontend + API + Postgres in one compose.
Eval & prompts	Golden transcript set; summary quality regression.	pytest + LLM-judge score or human rubric.
Rate limiting	Prevent API abuse.	Rate limit per org; embedding cache.

6. Sprint 6+ — Organizational Intelligence Product

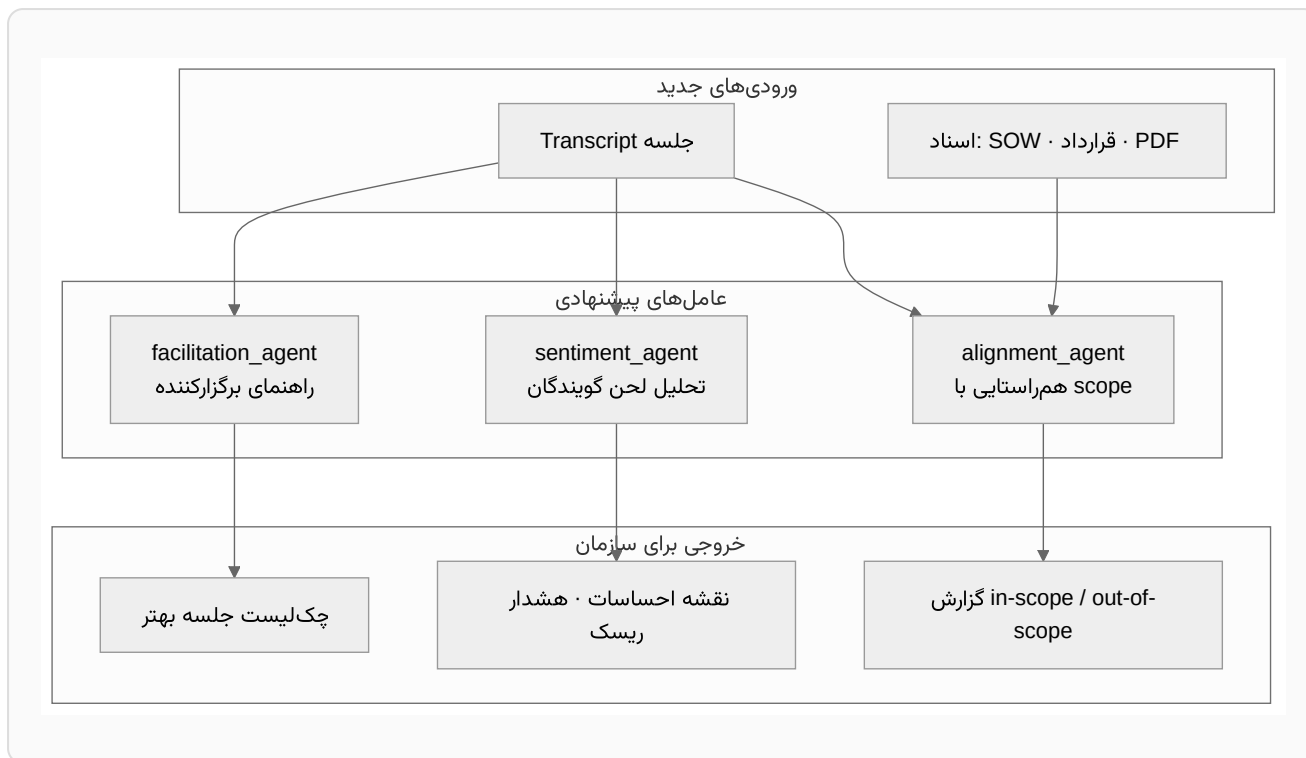
Goal: Competitive differentiation — beyond "summary + Jira."



Idea	Value
Decision tracking	Follow decisions across meetings — "Was decision X executed?"
Action items dashboard	All open tasks from all meetings, filter by assignee.
Meeting templates	Prompts tailored to standup / retro / planning.
Participation analysis	Talk ratio per speaker; one-sided meeting alerts.
Decision conflict	Agent comparing new meeting against prior decisions.
Follow-up suggestions	Auto-draft next agenda from open tasks.
Confluence / Notion	Publish minutes to org wiki.
Analysis versioning	Human edits on summary; re-run only part of pipeline.
Privacy	PII masking · retention · GDPR export.
On-prem	In-network deployment; optional self-hosted model.

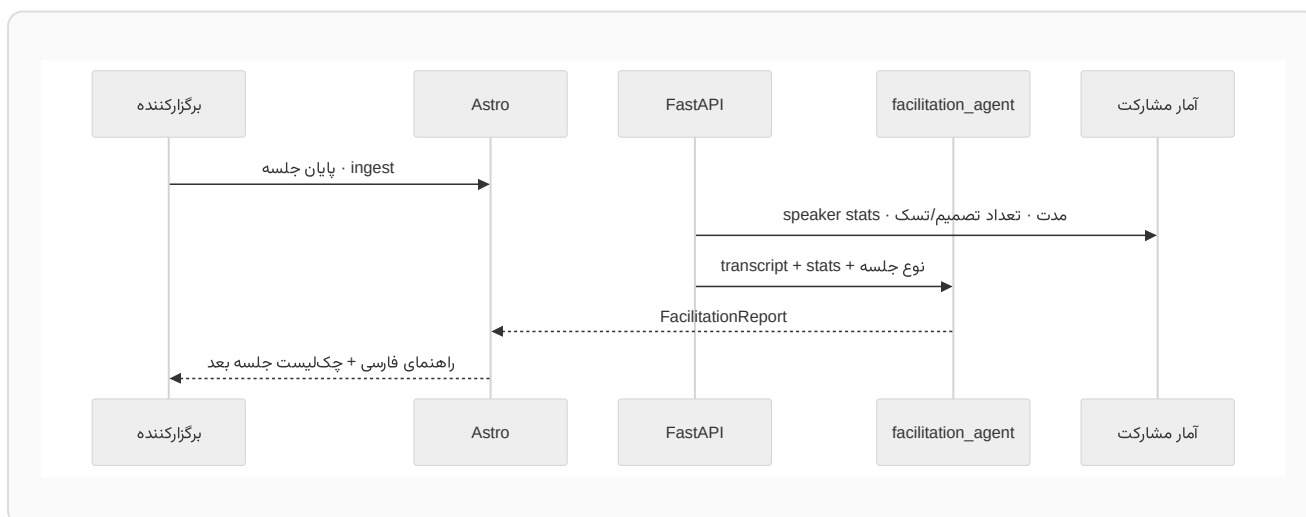
7. Advanced Capabilities (Differentiators)

Three capabilities turn the assistant from a note-taker into a meeting intelligence product. Each adds a new agent and, where needed, a new input.



7.1 Facilitator Guide (*implemented*)

After each meeting analysis, the system gives the **facilitator** practical, coaching-style suggestions: what went well, where time was wasted, and how to run the next session shorter and more effectively.



Manual checklist (today — without AI)

Before the meeting

- Define a one-line goal and expected output (decision / task / information).

- ❑ Send agenda with approximate time per item (timebox).
- ❑ Invite only essential people — each person's role should be clear.
- ❑ Have reference documents (SOW, contract) available in advance.

During the meeting

- ❑ State decisions with "We decided that..." and name the owner.
- ❑ Park off-topic discussions in a "parking lot."
- ❑ Every 15–20 minutes, give a 30-second progress recap.

After the meeting

- ❑ Publish summary + tasks within 24 hours.
- ❑ Schedule follow-up only if open tasks remain.

FacilitationReport output

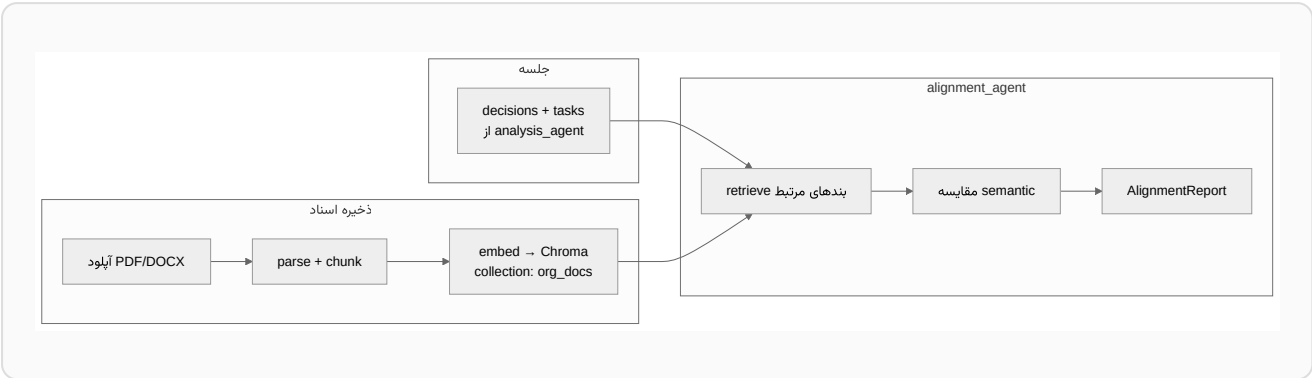
Field	Description
what_went_well	2–3 positives (e.g. clear decisions, balanced participation).
improvements	Specific suggestion (e.g. "Section X took 40% of time without a decision").
next_meeting_agenda	Draft agenda for next meeting from open tasks.
timebox_suggestion	Shorter duration or different format (e.g. async update).
coaching_summary	Short, encouraging summary for the facilitator.
facilitator_score	Optional 1–5 score based on rubric.

API: GET /api/meetings/{id}/facilitation — implemented, with "Meeting guide" tab in UI.

7.2 SOW / Contract Alignment (*planned*)

The organization can upload a **Statement of Work**, contract, or RFP. After each meeting, `alignment_agent` compares meeting decisions and commitments against

document clauses and labels each item **in-scope**, **out-of-scope**, or **needs legal/manager approval**.



Proposed data model

Entity	Fields
OrgDocument	id , title , type (sow contract rfp), project_key , uploaded_at
AlignmentFinding	meeting_decision , doc_excerpt , status (aligned out_of_scope unclear), confidence , doc_id
AlignmentReport	List of findings + Persian summary for project manager.

Proposed API

Method	Path	Action
POST	/api/documents	Upload SOW/contract · index in Chroma.
GET	/api/documents	List project documents.
POST	/api/meetings/{id}/alignment	Run alignment_agent · comparison report.

Product warning: AI output is not a substitute for legal counsel — for binding contracts, human review is mandatory. The UI must show a clear disclaimer.

7.3 Sentiment Analysis (*planned*)

On **transcript text** (not raw audio), per speaker and per meeting segment: overall tone (positive / neutral / negative), **misalignment** (e.g. positive words with hidden concern), and emotional trend through the meeting.

Transcript turns
per speaker



Chunk by speaker
or time window



sentiment_agent
Gemini structured output

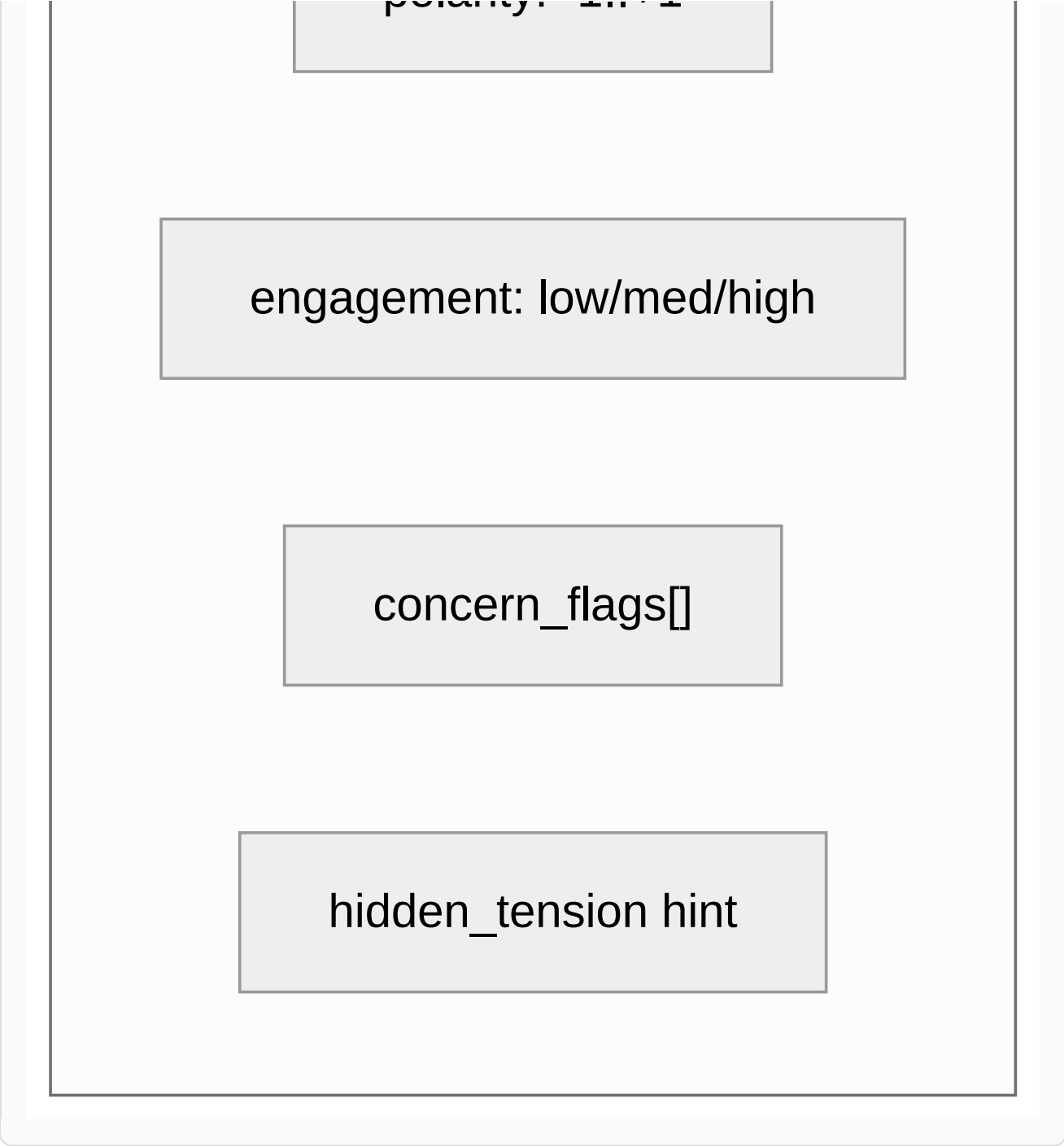


SpeakerSentimentProfile



متریک‌ها

polarity: -1 +1



SpeakerSentimentProfile output

Field	Meaning
speaker	Speaker name from transcript.
overall_tone	positive neutral negative mixed.
polarity_score	Numeric from -1 to +1.
concern_phrases	Short quotes suggesting concern or opposition.

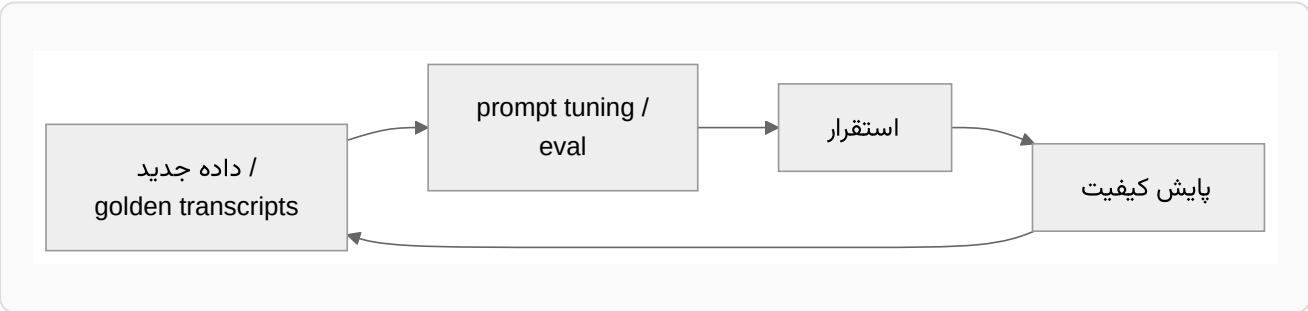
hidden_dissatisfaction	bool + explanation (e.g. "apparent agreement but emphasis on risk").
timeline	Tone shift at start / middle / end of meeting.

Proposed API: GET /api/meetings/{id}/sentiment — new tab beside summary/tasks/RAG.

Ethics & privacy: Sentiment analysis is sensitive for team management. Only admin/facilitator should see it; org opt-in required; no negative labeling in public reports. UI framing: "signal for follow-up conversation," not "personality judgment."

8. MLOps & Quality Cycle

The MVP does not fine-tune models — it uses a hosted LLM. Therefore the operational concern is not model training but **ensuring prompt and output quality over time** — exactly where an AI product quietly degrades without fanfare.



- **Golden set:** A curated set of transcripts with reference summaries and expected tasks.
- **Automated eval:** Score each new prompt/model version against the golden set with LLM-judge and/or human rubric, as part of CI.
- **Prompt versioning:** Treat prompts like versioned artifacts so a regression can be traced to a specific change.
- **Drift monitoring:** Track summary quality, RAG citation accuracy, and Jira conversion rate over time; alert on decline.

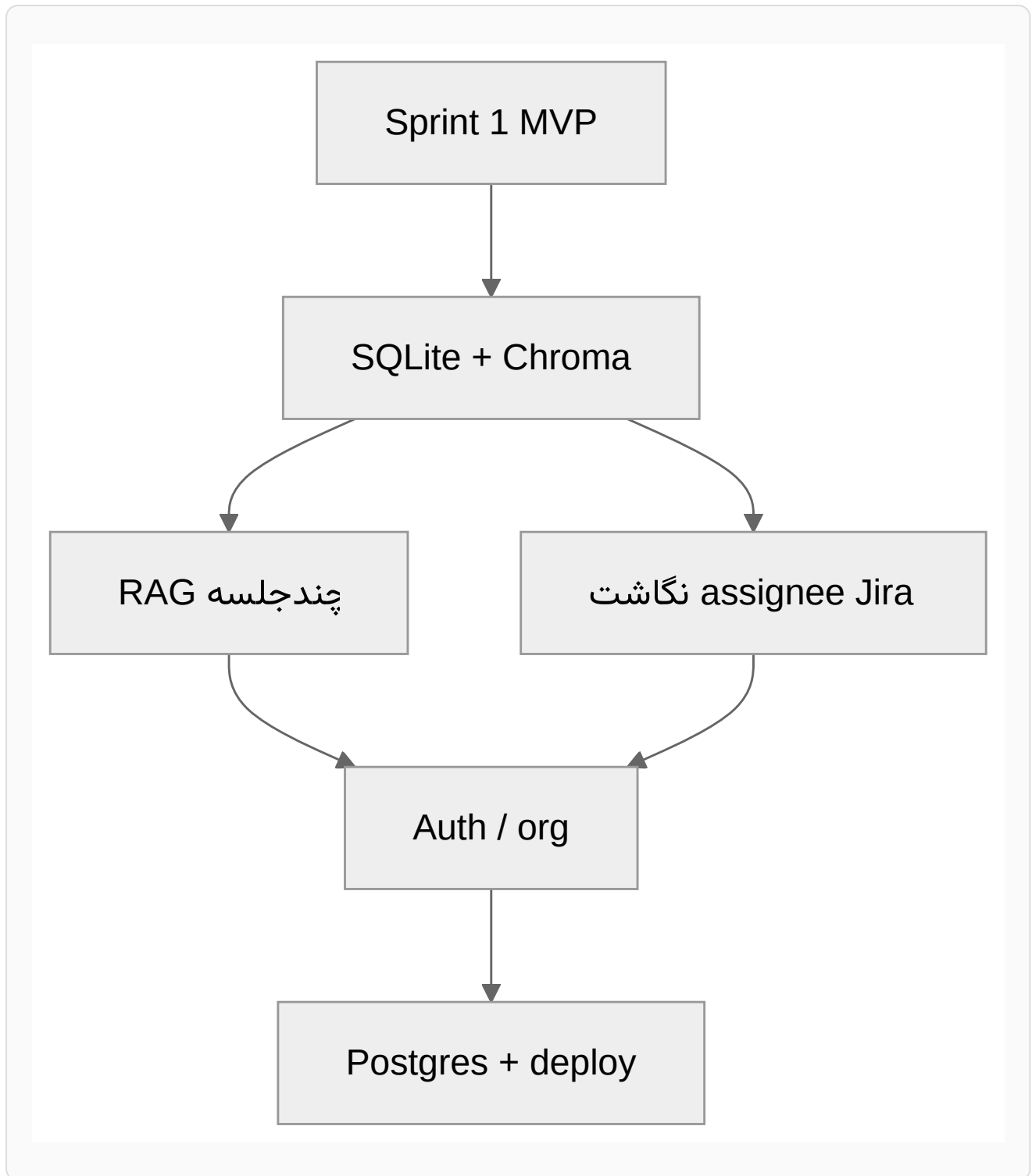
- **Logfire (initial implementation):** With [Pydantic Logfire](#), agent runs, FastAPI/HTTPX requests, and pipeline spans (ingest → analyze → index) are traceable. Enabled with `LOGFIRE_TOKEN` in `.env` ; without a token, overhead is negligible.

This work is scheduled with Sprint 5 (scale, quality, deployment) and is an honest counterpart to AIOps/MLOps theory: the lifecycle a product adopts as it matures, described realistically without overpromising in the MVP.

9. Prioritization

Priority	Item	Rationale
P0	Multi-meeting RAG + FTS archive	Highest value over MVP.
P0	Jira assignee mapping	Real pain for operational teams.
P1	Auth + workspace	Prerequisite for enterprise customer.
P1	Edit before Jira create	User trust in AI output.
P1	Facilitator guide (done) + deeper coaching	Immediate differentiation with light infra.
P1	SOW/contract alignment	Value for PMs and fixed-scope contracts.
P2	PostgreSQL + queue	When traffic exceeds demo scale.
P2	Calendar metadata + summary email	Reduce data-entry friction.
P2	Sentiment analysis	Only after RBAC and ethical opt-in.
P3	Multi-agent · Confluence	Long-term differentiation.

10. Technical Dependencies



The dependency graph explains sprint order: the storage foundation enables both multi-meeting RAG and assignee mapping; both motivate multi-tenancy (auth/org); and real tenant load eventually justifies PostgreSQL and production deployment.

11. Longer-Horizon Research Ideas

Beyond planned product work, several directions could anchor thesis research or further development:

- **Cross-meeting decision graph.** Model decisions as nodes and detect contradiction, reversal, and unfulfilled commitments across an organization's meeting history.
- **Multi-agent debate for summaries.** Use a critic/reviewer agent to challenge the first-pass summary, and measure whether adversarial review improves real faithfulness.
- **Semantic and speaker-aware chunking.** Compare turn-based vs semantic, speaker-aware chunking for retrieval accuracy on Persian transcripts.
- **Faithfulness evaluation for Persian RAG.** Build an evaluation harness for citation accuracy and hallucination rate, specific to low-resource meeting data.
- **Privacy-preserving analysis.** On-device or PII-masked pipelines so sensitive meetings can be analyzed without leaving the org network.
- **Human-in-the-loop quality.** Record human edits to summaries and tasks as feedback signals for prompt improvement and per-organization adaptation.

Related: [Project Overview & Vision](#) · [Technical Implementation](#).